第8讲作业：基于组件的软件可信性度量

刘小凡 10244602411

一、题目与数据

按照课件中的基于组件的软件可信性度量模型，计算游戏服务器系统在 $(\alpha, \beta) = (0.70, 0.30)$、$(0.60, 0.40)$、$(0.55, 0.45)$ 下的可信度量值和可信等级。其中 α、β 分别是关键组件组、非关键组件组的权重。

题目列出 13 个组件及其可信值：

组别	编号	组件	可信值
关键	CP1	网关组件	8.430
关键	CP2	游戏大厅组件	8.530
关键	CP3	公共组件	6.042
关键	CP4	配置组件	9.094
关键	CP5	核心服务器组件	8.289
非关键	CP6	总捕鱼游戏组件	6.192
非关键	CP7	用户捕鱼游戏组件	8.020
非关键	CP8	注册组件	7.984
非关键	CP9	游戏 AI 组件	8.713
非关键	CP10	后台管理组件	9.211
非关键	CP11	工具组件	7.777
非关键	CP12	消息组件	7.897
非关键	CP13	Maven 插件组件	8.075

分组说明：课件题目文字将非关键组件写为 CP6—CP12，但其后给出的非关键组件判断矩阵包含 CP6—CP13。这里依据该矩阵将 CP13 一并作为非关键组件，并保留全部 13 个组件参与计算。

1. 关键组件判断矩阵

	CP1	CP2	CP3	CP4	CP5
CP1	1	2	1/2	2	1/4
CP2	1/2	1	2	3	1/2
CP3	2	1/2	1	1	1/2
CP4	1/2	1/3	1	1	1/2
CP5	4	2	2	2	1

2. 非关键组件判断矩阵

	CP6	CP7	CP8	CP9	CP10	CP11	CP12	CP13
CP6	1	3	2	1/2	2	1	3	3
CP7	1/3	1	2	1	2	2	2	2
CP8	1/2	1/2	1	1/2	1	1/2	3	3
CP9	2	1	2	1	3	1/2	3	3
CP10	1/2	1/2	1	1/3	1	1	2	2
CP11	1	1/2	2	2	1	1	2	2
CP12	1/3	1/2	1	1/3	1/2	1/2	1	2
CP13	1/3	1/2	2	1/3	1/2	1/2	1/2	1

原始矩阵说明：课件将上表称为正互反判断矩阵，但表中 $a_{8,12}=3$、$a_{12,8}=1$，且 $a_{8,13}=3$、$a_{13,8}=2$；这两对数值的乘积不为 1，不满足正互反要求。以下程序直接代入课件原表，不擅自更改数据。非关键组权重及由其计算的系统值是按原始数据进行的暂定数值计算，严格的层次分析结论需要先确认或修正判断矩阵。

二、计算模型与方法

1. 组件权重

按照课件示例，分别使用右特征向量法（EV）、对数最小二乘法（LLSM）以及课件给出的卡方最小二乘法迭代程序（CSM）计算每组权重，再比较强度指标：

$$TD=\sum_{i=1}^{n}\sum_{j=1}^{n}\left|a_{ij}-\frac{w_i}{w_j}\right|。$$

选取本组 TD 最小的权重向量。LLSM 使用每行几何平均数并归一化：

$$g_i=\left(\prod_{j=1}^{n}a_{ij}\right)^{1/n},\qquad w_i=\frac{g_i}{\sum_{k=1}^{n}g_k}。$$

CSM 按第 6 讲课件第 49—51 页的迭代代码，从均匀权重开始，计算

$$d_i=\sum_{j=1}^{n}\left[(1+a_{ji}^2)\frac{w_i}{w_j}-(1+a_{ij}^2)\frac{w_j}{w_i}\right]。$$

选择 $|d_i|$ 最大的下标 m，计算

$$U=\sum_{j\neq m}(1+a_{mj}^2)\frac{w_j}{w_m},\qquad D=\sum_{j\neq m}(1+a_{jm}^2)\frac{w_m}{w_j},$$

将 w_m 乘以 $\sqrt{U/D}$ 并归一化，重复至 $\max_i|d_i|\le 10^{-10}$。对非互反的原始矩阵，程序仍按课件步骤进行数值代入，结果的适用限制见上面的说明。

2. 系统可信度量模型

设 FC_i 为关键组件可信值，组内权重为 α_i；NFC_j 为非关键组件可信值，组内权重为 β_j。课件的系统可信度量模型为：

$$T_S=\alpha\left(\prod_{i=1}^{5}FC_i^{\alpha_i}\right)+\beta\left(\prod_{j=1}^{8}NFC_j^{\beta_j}\right),\qquad \alpha+\beta=1。$$

即先分别计算关键组件组、非关键组件组的加权乘积，再用给定的 α、β 加权求和。

三、组件权重与系统可信值

1. 组件权重的选择

判断矩阵	EV 的 TD	LLSM 的 TD	CSM 的 TD	按最小 TD 选择
关键组件	11.411122	**11.376383**	11.476580	LLSM
非关键组件（原表）	23.238833	22.908112	**21.965839**	CSM

程序使用未四舍五入的权重计算 TD 和系统可信值。按上述数值选择，组内权重为：

组件	组内权重	组件	组内权重
CP1	0.160206	CP6	0.196399
CP2	0.199574	CP7	0.146706
CP3	0.160206	CP8	0.094936
CP4	0.111956	CP9	0.190893
CP5	0.368057	CP10	0.096863
—	—	CP11	0.150190
—	—	CP12	0.063709
—	—	CP13	0.060304

2. 系统可信值

两个组件组的加权乘积为：

$$T_{\text{关键组}} = \prod_{i=1}^{5} FC_i^{\alpha_i} \approx 8.029095, \qquad T_{\text{非关键组}} = \prod_{j=1}^{8} NFC_j^{\beta_j} \approx 7.804853。$$

例如，$(\alpha, \beta) = (0.70, 0.30)$ 时：

$$T_S = 0.70 \times T_{\text{关键组}} + 0.30 \times T_{\text{非关键组}} \approx 7.961822。$$

α	β	系统可信值 T_S	满足课件 III 级属性条件时	不满足该属性条件时
0.70	0.30	7.961822	III 级	II 级
0.60	0.40	7.939398	III 级	II 级
0.55	0.45	7.928186	III 级	II 级

上述数值以非关键组件判断矩阵原表为前提；若课件数据得到更正，应重新计算权重及系统可信值。

3. 等级判断

课件中的关键组件数量限制为 $n - \lceil 2n/3 \rceil$。本题 $n = 5$，因此允许低于相应门槛的关键组件最多为：

$$5 - \lceil 5 \times 2/3 \rceil = 1。$$

三组系统可信值都在 $[7.0, 8.5)$ 内。关键组件中低于 7.0 的只有 CP3（6.042），共 1 个，满足 III 级的组件数量条件；全部 13 个组件的可信值也都高于 4.5。

但课件 III 级的第二项写的是"没有低于 4.5 分的**可信属性**"，而本题只列出了组件可信值，没有给出逐项可信属性值。组件可信值均大于 4.5 不能直接证明所有属性值都大于或等于 4.5。因此：

- 如果所有可信属性值均不低于 4.5，则三组均为 **III 级**；若老师在本题中将该条件按组件可信值判断，也得到 III 级。
- 如果存在低于 4.5 的可信属性，则不满足 III 级。此时低于 4.5 的关键组件有 0 个，且 $T_S \geq 4.5$，按课件向下判断均为 **II 级**。

四、C++ 程序

程序内置课件数据，计算三种候选权重和 TD，选择组内权重，计算三组系统值，并分别给出满足和不满足 III 级属性条件时的等级。程序还会检查原矩阵中不互反的数值。运行后按回车退出，方便截图。

```
g++ -std=c++17 -O2 -Wall -Wextra -pedantic -static 第8讲作业.cpp -o 第8讲作业.exe
.\第8讲作业.exe
```

```cpp
#include <algorithm>
#include <cmath>
#include <iomanip>
#include <iostream>
#include <numeric>
#include <stdexcept>
#include <string>
#include <vector>

using namespace std;
using Vector = vector<double>;
using Matrix = vector<Vector>;

// 课件各权重求法都将权重向量归一化。
Vector normalize(Vector w) {
    double sum = accumulate(w.begin(), w.end(), 0.0);
    for (double& x : w) x /= sum;
    return w;
}

Vector multiply(const Matrix& a, const Vector& w) {
    Vector result(w.size(), 0.0);
    for (size_t i = 0; i < w.size(); ++i)
        for (size_t j = 0; j < w.size(); ++j)
            result[i] += a[i][j] * w[j];
    return result;
}

// EV：幂迭代求最大特征值对应的右特征向量。
Vector eigenvectorMethod(const Matrix& a) {
    const size_t n = a.size();
    Vector w(n, 1.0 / n);
    for (int k = 0; k < 100000; ++k) {
        Vector next = normalize(multiply(a, w));
        double delta = 0.0;
        for (size_t i = 0; i < n; ++i) delta = max(delta, abs(next[i] - w[i]));
        w = next;
        if (delta < 1e-13) return w;
    }
    throw runtime_error("EV did not converge");
}

// LLSM：各行的几何平均数归一化。
Vector logarithmicLeastSquares(const Matrix& a) {
    const size_t n = a.size();
    Vector result(n, 1.0);
    for (size_t i = 0; i < n; ++i) {
        for (size_t j = 0; j < n; ++j) result[i] *= a[i][j];
```

```
        result[i] = pow(result[i], 1.0 / n);
    }
    return normalize(result);
}

// 按第6讲课件第49–51页的 CSM 迭代代码计算。
Vector chiSquareMethod(const Matrix& a) {
    const size_t n = a.size();
    Vector w(n, 1.0 / n);
    for (int k = 0; k < 100000; ++k) {
        size_t m = 0;
        double largest = -1.0;
        for (size_t i = 0; i < n; ++i) {
            double d = 0.0;
            for (size_t j = 0; j < n; ++j) {
                d += (1 + a[j][i] * a[j][i]) * w[i] / w[j]
                   - (1 + a[i][j] * a[i][j]) * w[j] / w[i];
            }
            if (abs(d) > largest) {
                largest = abs(d);
                m = i;
            }
        }
        if (largest <= 1e-10) return w;
        double up = 0.0, down = 0.0;
        for (size_t j = 0; j < n; ++j) {
            if (j == m) continue;
            up += (1 + a[m][j] * a[m][j]) * w[j] / w[m];
            down += (1 + a[j][m] * a[j][m]) * w[m] / w[j];
        }
        w[m] *= sqrt(up / down);
        w = normalize(w);
    }
    throw runtime_error("CSM did not converge");
}

double totalDeviation(const Matrix& a, const Vector& w) {
    double td = 0.0;
    for (size_t i = 0; i < w.size(); ++i)
        for (size_t j = 0; j < w.size(); ++j)
            td += abs(a[i][j] - w[i] / w[j]);
    return td;
}

struct WeightResult {
    string method;
    Vector w;
    double td;
```

```
};

vector<WeightResult> allWeights(const Matrix& a) {
    vector<WeightResult> result = {
        {"EV", eigenvectorMethod(a), 0.0},
        {"LLSM", logarithmicLeastSquares(a), 0.0},
        {"CSM", chiSquareMethod(a), 0.0}
    };
    for (WeightResult& r : result) r.td = totalDeviation(a, r.w);
    return result;
}

const WeightResult& bestByTD(const vector<WeightResult>& results) {
    return *min_element(results.begin(), results.end(),
        [](const WeightResult& x, const WeightResult& y) { return x.td < y.td; });
}

double geometricGroupScore(const Vector& values, const Vector& w) {
    double result = 1.0;
    for (size_t i = 0; i < values.size(); ++i) result *= pow(values[i], w[i]);
    return result;
}

int below(const Vector& values, double threshold) {
    return static_cast<int>(count_if(values.begin(), values.end(),
        [threshold](double value) { return value < threshold; }));
}

// 属性值题目未提供，因此用参数分别计算满足和不满足属性条件的等级。
int grade(double t, const Vector& critical, const Vector& all,
          bool allAttributesAtLeast45) {
    const int n = static_cast<int>(critical.size());
    const int maxLow = n - (2 * n + 2) / 3;  // n - ceil(2n/3)
    if (t >= 9.5 && below(critical, 9.5) <= maxLow && below(all, 8.5) == 0) return 5;
    if (t >= 8.5 && below(critical, 8.5) <= maxLow && below(all, 7.0) == 0) return 4;
    if (t >= 7.0 && below(critical, 7.0) <= maxLow && allAttributesAtLeast45) return 3;
    if (t >= 4.5 && below(critical, 4.5) <= maxLow) return 2;
    return 1;
}

void printTD(const vector<WeightResult>& results) {
    for (const WeightResult& r : results) cout << ' ' << r.method << '=' << r.td;
    cout << '\n';
}

void printWeights(const Vector& w, int first) {
    for (size_t i = 0; i < w.size(); ++i) {
        cout << "CP" << first + static_cast<int>(i) << '=' << w[i];
```

```
        if (i + 1 == w.size() || (i + 1) % 4 == 0) cout << '\n';
        else cout << "  ";
    }
}

void printNonreciprocal(const Matrix& a, int first) {
    for (size_t i = 0; i < a.size(); ++i) {
        for (size_t j = i + 1; j < a.size(); ++j) {
            if (abs(a[i][j] * a[j][i] - 1.0) > 1e-10) {
                cout << "CP" << first + static_cast<int>(i)
                     << "/CP" << first + static_cast<int>(j)
                     << ": " << a[i][j] << " * " << a[j][i] << " != 1\n";
            }
        }
    }
}

int main() {
    // 关键组件 CP1–CP5 的判断矩阵。
    const Matrix criticalMatrix = {
        {1, 2, 1.0/2, 2, 1.0/4},
        {1.0/2, 1, 2, 3, 1.0/2},
        {2, 1.0/2, 1, 1, 1.0/2},
        {1.0/2, 1.0/3, 1, 1, 1.0/2},
        {4, 2, 2, 2, 1}
    };
    // 非关键组件 CP6–CP13，保留课件原始矩阵。
    const Matrix noncriticalMatrix = {
        {1, 3, 2, 1.0/2, 2, 1, 3, 3},
        {1.0/3, 1, 2, 1, 2, 2, 2, 2},
        {1.0/2, 1.0/2, 1, 1.0/2, 1, 1.0/2, 3, 3},
        {2, 1, 2, 1, 3, 1.0/2, 3, 3},
        {1.0/2, 1.0/2, 1, 1.0/3, 1, 1, 2, 2},
        {1, 1.0/2, 2, 2, 1, 1, 2, 2},
        {1.0/3, 1.0/2, 1, 1.0/3, 1.0/2, 1.0/2, 1, 2},
        {1.0/3, 1.0/2, 2, 1.0/3, 1.0/2, 1.0/2, 1.0/2, 1}
    };
    const Vector criticalValues = {8.430, 8.530, 6.042, 9.094, 8.289};
    const Vector noncriticalValues = {6.192, 8.020, 7.984, 8.713,
                                      9.211, 7.777, 7.897, 8.075};
    Vector allValues = criticalValues;
    allValues.insert(allValues.end(), noncriticalValues.begin(), noncriticalValues.end());

    const auto criticalResults = allWeights(criticalMatrix);
    const auto noncriticalResults = allWeights(noncriticalMatrix);
    const auto& cw = bestByTD(criticalResults);
    const auto& nw = bestByTD(noncriticalResults);
    const double criticalScore = geometricGroupScore(criticalValues, cw.w);
```

```
    const double noncriticalScore = geometricGroupScore(noncriticalValues, nw.w);

    cout << fixed << setprecision(6);
    cout << "Critical group TD:"; printTD(criticalResults);
    cout << "Selected: " << cw.method << '\n';
    printWeights(cw.w, 1);
    cout << "Noncritical group TD:"; printTD(noncriticalResults);
    cout << "Selected: " << nw.method << '\n';
    printWeights(nw.w, 6);
    cout << "Critical score: " << criticalScore << '\n';
    cout << "Noncritical score: " << noncriticalScore << '\n';
    cout << "\nalpha  beta    T          Grade if attributes >= 4.5 / not\n";
    for (double alpha : {0.70, 0.60, 0.55}) {
        const double beta = 1.0 - alpha;
        const double t = alpha * criticalScore + beta * noncriticalScore;
        cout << setprecision(2) << alpha << "   " << beta << "   "
             << setprecision(6) << t << "   "
             << grade(t, criticalValues, allValues, true) << " / "
             << grade(t, criticalValues, allValues, false) << '\n';
    }
    cout << "\nNonreciprocal pairs in the original matrix:\n";
    printNonreciprocal(noncriticalMatrix, 6);
    cout << "Results require confirmation of the matrix and attributes.\n";
    cout << "\nPress Enter to exit..." << flush;
    cin.get();
    return 0;
}
```

实际运行结果

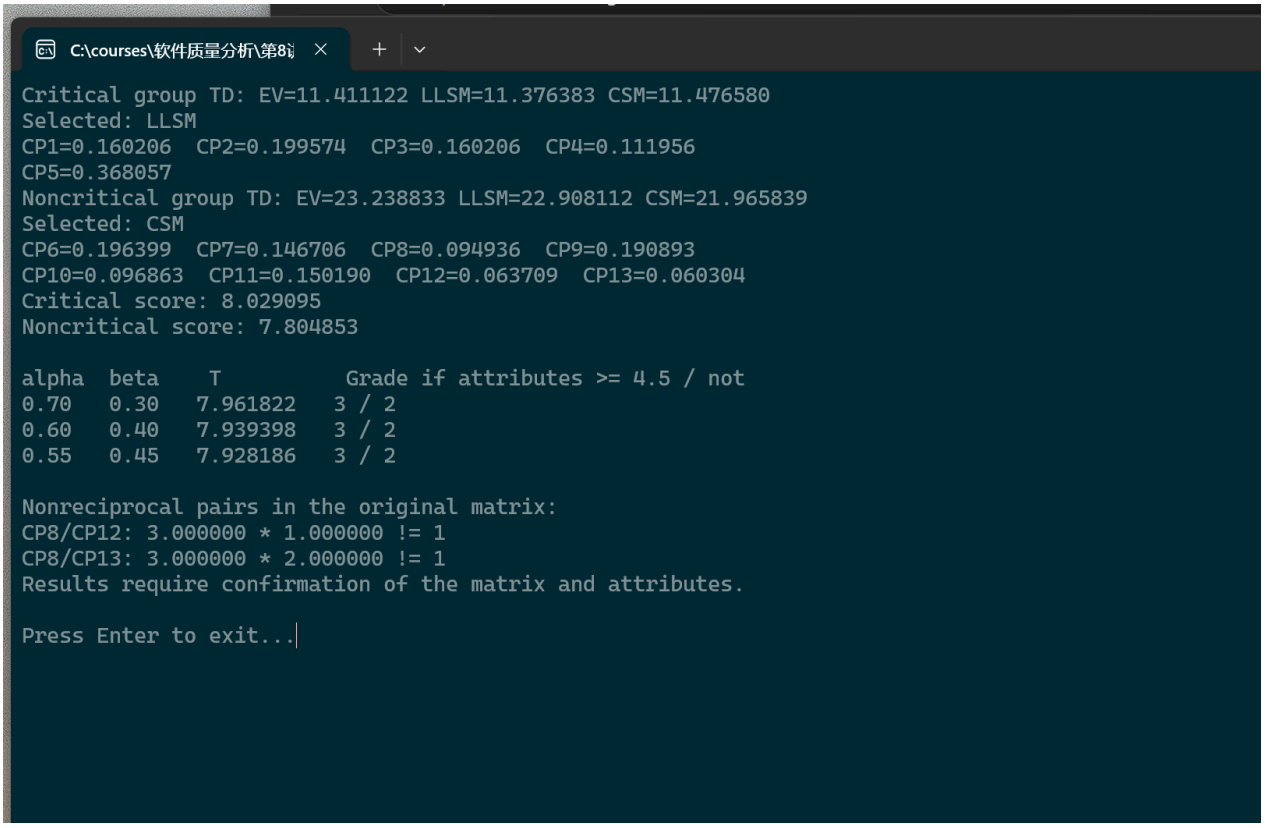
```
Critical group TD: EV=11.411122 LLSM=11.376383 CSM=11.476580
Selected: LLSM
CP1=0.160206  CP2=0.199574  CP3=0.160206  CP4=0.111956
CP5=0.368057
Noncritical group TD: EV=23.238833 LLSM=22.908112 CSM=21.965839
Selected: CSM
CP6=0.196399  CP7=0.146706  CP8=0.094936  CP9=0.190893
CP10=0.096863  CP11=0.150190  CP12=0.063709  CP13=0.060304
Critical score: 8.029095
Noncritical score: 7.804853

alpha  beta    T          Grade if attributes >= 4.5 / not
0.70   0.30   7.961822   3 / 2
0.60   0.40   7.939398   3 / 2
0.55   0.45   7.928186   3 / 2

Nonreciprocal pairs in the original matrix:
CP8/CP12: 3.000000 * 1.000000 != 1
CP8/CP13: 3.000000 * 2.000000 != 1
Results require confirmation of the matrix and attributes.

Press Enter to exit...
```

五、运行截图


C:\courses\软件质量分析\第8
Critical group TD: EV=11.411122 LLSM=11.376383 CSM=11.476580
Selected: LLSM
CP1=0.160206 CP2=0.199574 CP3=0.160206 CP4=0.111956
CP5=0.368057
Noncritical group TD: EV=23.238833 LLSM=22.908112 CSM=21.965839
Selected: CSM
CP6=0.196399 CP7=0.146706 CP8=0.094936 CP9=0.190893
CP10=0.096863 CP11=0.150190 CP12=0.063709 CP13=0.060304
Critical score: 8.029095
Noncritical score: 7.804853
alpha beta T Grade if attributes >= 4.5 / not
0.70 0.30 7.961822 3 / 2
0.60 0.40 7.939398 3 / 2
0.55 0.45 7.928186 3 / 2
Nonreciprocal pairs in the original matrix:
CP8/CP12: 3.000000 * 1.000000 != 1
CP8/CP13: 3.000000 * 2.000000 != 1
Results require confirmation of the matrix and attributes.
Press Enter to exit...